NeRF Architectures under Sparse-View Conditions: A Systematic Benchmarking Methodology

Felipe G. Silva*, Leandro A. Neves*, Guilherme F. Roberto*, Thiago Pozati*, Domingos Oliveira†,‡, Adriano B. Silva‡, Thaína A. A. Tosta§ and Marcelo Z. do Nascimento‡
*Department of Computer Science and Statistics (DCCE). São Paulo State University (UNESP), São José do Rio Preto, SP, Brazil. Email: felipe.gomes-silva@unesp.br
†Federal Institute of Education, Science and Technology of São Paulo (IFSP), São Paulo, SP, Brazil.
‡Faculty of Computer Science (FACOM), Federal University of Uberlândia (UFU), Uberlândia, MG, Brazil
§Department of Computing and Mathematics, University of São Paulo (USP), Ribeirão Preto, SP, Brazil

***Abstract*—This work proposes a methodology for systematically evaluating the trade-offs between computational performance and reconstruction fidelity of Neural Radiance Field (NeRF) architectures under sparse-view conditions. Six representative methods are evaluated using visual fidelity metrics, including Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM), and Learned Perceptual Image Patch Similarity (LPIPS), together with computational performance metrics, including training convergence time, Video Random Access Memory (VRAM) footprint, and inference latency. The experimental results reveal a clear trade-off between reconstruction fidelity and computational efficiency. Per-Scene models consistently outperform Generalizable models in visual quality, achieving up to 27.11 dB PSNR, 0.875 SSIM, and 0.171 LPIPS, while also providing significantly higher inference throughput and lower memory consumption. In contrast, generalizable models offer greater flexibility for acquisition-constrained scenarios but exhibit limited reconstruction fidelity and unstable optimization under severe sparse-view conditions. The proposed methodology and the resulting quantitative analyses establish a standardized and reproducible framework for benchmarking heterogeneous neural rendering architectures, enabling systematic model selection according to computational and acquisition constraints.**

I. Introduction

Neural Radiance Fields (NeRFs) have established themselves as the state of the art for novel view synthesis, enabling high-fidelity reconstruction of three-dimensional scenes from multi-view observations across a broad range of computer vision and computer graphics applications [1]–[3]. Nevertheless, most neural rendering techniques require substantial computational resources and densely sampled multi-view training data, which restricts their applicability to resource-constrained scenarios such as mobile scanning, virtual try-on systems, and sparse-view reconstruction.

Recent advances in neural rendering have focused on improving the computational efficiency and generalization capability of NeRF-based methods. Representative approaches include Instant Neural Graphics Primitives (Instant-NGP), Nerfacto, PixelNeRF, and Generalizable NeRF Transformer (GNT) [2], [4]–[6]. These methods explore different design strategies, including efficient spatial encoding mechanisms [4], hybrid architectures that integrate Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) [6]–[8], and model pre-computation (baking) into compact representations with reduced memory requirements [9]. Although these developments have significantly improved different aspects of neural rendering, their computational characteristics and reconstruction performance have generally been evaluated independently or under different experimental settings.

Consequently, a systematic and reproducible comparison of generalizable and per-scene neural rendering architectures under standardized sparse-view protocols remains unavailable. Existing studies have primarily investigated NeRF models in limited-data scenarios [10], [11], whereas benchmarking frameworks such as NerfBaselines [12] focus predominantly on traditional and per-scene methods. This hinders a comprehensive characterization of the trade-offs between computational efficiency and reconstruction fidelity across different neural rendering paradigms.

To address this limitation, a systematic evaluation framework for neural rendering architectures under sparse-view conditions is proposed here. Reconstruction fidelity was quantified using three widely adopted image quality metrics, namely Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM) [13], and Learned Perceptual Image Patch Similarity (LPIPS) [14]. Computational performance was quantified through complementary system-level metrics, including training convergence time, inference latency, and Video Random Access Memory (VRAM) consumption, enabling a quantitative characterization of the trade-offs between computational cost and reconstruction quality. Thus, the main contributions of this work are summarized as follows:

- A unified evaluation protocol was defined for generalizable and per-scene NeRF architectures, ensuring reproducibility through standardized data partitions, controlled execution environments, and consistent evaluation metrics across three sparse-view data regimes.
- Integration wrappers extending the Nerfstudio framework with PixelNeRF and GNT and five respective Fine Tuning strategies for each of them were implemented to provide a unified experimental infrastructure for standardized benchmarking of heterogeneous neural rendering architectures. Consequently, these modules were publicy released, enabling the deployment of these models across

broader applications.

II. Theoretical Background

A. Neural Radiance Fields and Volumetric Rendering

Neural Radiance Fields (NeRFs) model continuous three-dimensional scene geometry and view-dependent appearance by mapping spatial coordinates and viewing directions to volumetric density and emitted radiance using Multilayer Perceptrons (MLPs) [15]. Given a spatial position $\mathbf{x} = (x, y, z)$ and a unit viewing direction $\mathbf{d} = (\theta, \phi)$, the network outputs differential volume density $\sigma(\mathbf{x}) \in \mathbb{R}_{\geq 0}$ and directional color $\mathbf{c} \in [0, 1]^3$:

$$\gamma(\mathbf{x}, \mathbf{d}) \to \mathbf{v}, \quad F'_\theta(\mathbf{v}) \to (\sigma, \mathbf{c}) \tag{1}$$

where θ denotes the trainable network parameters, F'_θ represents the MLP, and γ is a positional encoding mapping coordinates into a higher-dimensional space $\mathbf{v}$ through sinusoidal basis functions, enabling the network to represent high-frequency geometric and textural variations [15].

Synthesizing the color observed by a camera pixel involves casting a ray $\mathbf{r}(t) = \mathbf{o} + t\mathbf{d}$ between near and far bounds (t_n, t_f) and evaluating the volumetric rendering integral [15]:

$$\hat{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\sigma(\mathbf{r}(t))\mathbf{c}(\mathbf{r}(t), \mathbf{d})\, dt \tag{2}$$

where $T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\, ds\right)$ denotes the accumulated transmittance along the ray. Numerical approximation evaluates this continuous integral through stratified quadrature sampling, compositing discrete alpha-blended color samples along each sampled ray.

B. Architectural Optimizations and Generalization

Standard coordinate-based MLPs achieve remarkable visual fidelity, yet their dense parameter evaluations impose computational and convergence bottlenecks. Accelerating this optimization has motivated hybrid spatial representations that replace global positional encodings with explicit multiresolution data structures. Instant-NGP maps spatial coordinates to multiscale hash tables of trainable feature vectors, reducing convergence times from hours to seconds [4]. Nerfacto builds upon this paradigm by coupling multiresolution hash grids with proposal sampling, camera pose refinement, spherical harmonics, and appearance embeddings, achieving a balanced trade-off between rendering speed and perceptual quality [2].

These per-scene representations, however, rely on dense multi-view coverage with substantial angular overlap. Practical applications, ranging from mobile capture and robotic navigation to biomedical imaging, such as 3D histology and holotomography, routinely face data acquisition constraints where only a few sparse views are available [16]. Under such sparse regimes, per-scene optimization becomes heavily underconstrained, possibly resulting in geometric degradation, false surfaces, and floating artifacts [10], [17].

To overcome these data limitations, generalizable NeRFs learn inductive structural and multi-view priors across large scene collections, enabling feed-forward synthesis on novel target environments [5], [6], [18], [19]. PixelNeRF projects query 3D points onto 2D image planes, extracting spatial feature vectors via a convolutional backbone (ResNet) to condition a coordinate-based radiance MLP [5]. Addressing the locality limits of convolutional operators, the Generalizable NeRF Transformer (GNT) replaces the CNN backbone with a Vision Transformer (ViT) and substitutes the terminal MLP with a cross-attention transformer, aggregating multi-view features directly along epipolar lines [6].

C. Fine-Tuning Strategies and Parameter-Efficient Regimes in NeRFs

Pre-trained generalizable models capture rich geometric and cross-view priors during large-scale pre-training [5], [6]. Deploying these networks directly in a zero-shot (ZS) regime on novel real-world scenes exposes them to pronounced domain shifts caused by discrepancies in sensor intrinsics, illumination conditions, and complex out-of-distribution textures [10], [20]. In sparse-view settings ($N \leq 10$), feed-forward zero-shot predictions frequently suffer from spatial ambiguities and lack high-frequency details, exhibiting noticeable isotropic blurring [5], [18].

To bridge this domain gap, generalizable architectures employ fine-tuning (FT), optimizing network weights directly on the sparse target observations using the photometric reconstruction loss [5], [18]:

$$\mathcal{L}_{\text{photo}} = \frac{1}{|\mathcal{R}|} \sum_{\mathbf{r} \in \mathcal{R}} \|\hat{C}(\mathbf{r}) - C(\mathbf{r})\|_2^2 \tag{3}$$

where $\mathcal{R}$ denotes a batch of synthesized rays and $C(\mathbf{r})$ represents the observed ground-truth pixel color.

Nevertheless, unconstrained gradient descent under view sparsity constitutes an ill-posed inverse problem [10], [17], [21]. When high-capacity architectures are optimized against sparse camera rays without dense geometric constraints, the representation becomes susceptible to representational collapse [10], [17]. The network easily overfits to ray-specific photometric objectives by creating degenerate density floaters near camera centers, underperforming in global 3D consistency and degrading novel view synthesis quality [10], [21].

This tension between model adaptation capacity and optimization stability parallels the challenges encountered in fine-tuning deep learning models across language, vision, and multimodal domains under scarce downstream supervision [22]–[24]. In their comprehensive benchmarking study of foundation model adaptation, Gupta et al. demonstrate that updating all model weights under limited target data destabilizes pre-trained representations, increases optimizer memory overhead, and can lead to overfitting. To address these vulnerabilities, the same authors formalize a comparative taxonomy comprising full model tuning, modular parameter freezing, selective parameter updates, and low-rank reparameterization. Transposing this modular methodology to generalizable neural rendering yields five distinct FT strategies [24]:

1) Full Fine-Tuning (Full-FT): All network parameters $\Theta = \{W, b\}$ across the feature extraction backbone and the volumetric rendering network are simultaneously updated via backpropagation. Although offering unconstrained representational capacity, Full-FT requires maintaining full optimizer moment tensors for every weight, incurring substantial memory overhead and heightening vulnerability to representational collapse when supervised by sparse views [10], [24].

2) Partial Fine-Tuning: Decoder (PFT-Decoder): Gradient updates are restricted exclusively to the downstream volumetric rendering head—the cross-attention transformer in GNT [6] or the radiance MLP in PixelNeRF [5]—while the spatial feature extraction backbone remains frozen. Freezing the 2D perceptual layers preserves the generalizable representations established during pre-training, directing gradient updates solely toward refining multi-view radiance aggregation and scene-specific illumination [24].

3) Partial Fine-Tuning: Backbone (PFT-Backbone): Inverting the modular restriction, PFT-Backbone freezes the volumetric rendering head and updates solely the 2D feature extractor (the ResNet in PixelNeRF or the shallow patch embedder in GNT). This configuration enables the model to align its low-level perceptual feature maps with out-of-distribution textures and sensor profiles, leaving the pre-trained volumetric ray-marching logic undisturbed.

4) Selective PEFT - BitFit: BitFit isolates and optimizes exclusively the additive bias vectors $b \in \mathbb{R}^{d_{\text{out}}}$ of linear and convolutional transformations, keeping all projection weight matrices $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ strictly frozen [23]:

$$y = Wx + (b + \Delta b) \tag{4}$$

As demonstrated in foundation model benchmarks by Ben Zaken et al. [23] and Gupta et al. [24], modifying solely the additive biases (typically representing less than 0.5% of the total parameter count) acts as an effective structural regularizer. This parameter isolation stabilizes the optimization trajectory, preventing representational drift while providing sufficient flexibility to adjust activation thresholds for target scene alignment.

5) Additive PEFT: Low-Rank Adaptation (PEFT-LoRA): LoRA freezes pre-trained weight matrices $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and parameterizes weight increments through a low-rank decomposition [22]:

$$W' = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA \tag{5}$$

where $B \in \mathbb{R}^{d_{\text{out}} \times r}$ and $A \in \mathbb{R}^{r \times d_{\text{in}}}$ are low-rank factor matrices with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$, and α is a constant scaling hyperparameter. Matrix A is initialized from a Gaussian distribution and B is initialized to zero, ensuring $\Delta W = 0$ at the onset of adaptation. Restricting updates to an intrinsic low-dimensional subspace drastically curtails trainable parameters and optimizer memory during fine-tuning. Furthermore, as observed by Gupta et al. [24], low-rank constraints act as an implicit regularizer that prevents catastrophic collapse when adapting deep foundational architectures to scarce downstream data.

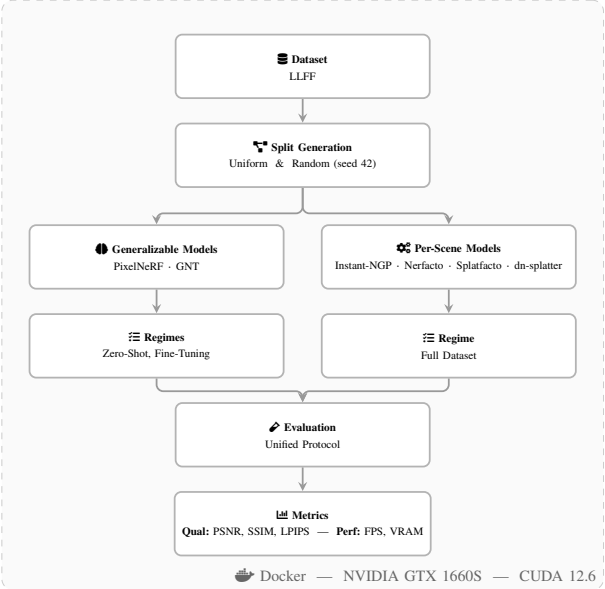


Fig. 1: Overview of the proposed methodology. The pipeline comprises dataset preparation, sparse-view sampling, model configuration and training under distinct learning paradigms, followed by a unified evaluation protocol that systematically characterizes the trade-offs between computational performance and reconstruction fidelity.

Understanding the interaction between these modular fine-tuning strategies, generalizable priors, and per-scene representations requires a unified, reproducible benchmarking protocol, as formalized in the subsequent section.

III. Materials and Methods

This section presents the proposed methodology for systematically evaluating the trade-offs between computational performance and reconstruction fidelity of NeRF models under sparse-view conditions. Figure 1 illustrates the proposed evaluation pipeline, which consists of three main stages: (i) dataset selection and sparse-view sampling; (ii) configuration and training of the selected neural rendering architectures according to their respective learning paradigms; and (iii) quantitative evaluation using standardized metrics to jointly characterize computational efficiency and reconstruction fidelity.

A. Dataset & Splits

The experiments were conducted on the Local Light Field Fusion (LLFF) dataset, which comprises 8 real-world scenes captured with a forward-facing moving camera. Camera poses and depth bounds were obtained through COLMAP, following standard dataset procedures [20]. Furthermore, images were evaluated using downscale of 8 resolution (504×378).

The complete data splitting protocol follows the few-shot evaluation configuration established by Niemeyer et al. [10]. Specifically, the testing split was generated by reserving images with indices that are multiples of 8 (llffhold=8), while the training views for the sparse regimes were strictly limited to

$N \in \{3, 6, 10\}$. The selection of these training views was performed using two distinct strategies: (i) uniform, where views are sampled via linear interpolation over the dataset indices, distributing them evenly along the camera trajectory (a strategy that does not generalize to other datasets, since LLFF indices encode camera trajectory order); and (ii) random, where views are randomly selected using a fixed seed (42) to ensure reproducibility. Comparing these two strategies assesses the models sensitivity to the spatial distribution of the input views, as demonstrated in Table I. Given the empirical advantage of uniform sampling (+0.54 dB PSNR on the fern scene), this strategy was adopted for all subsequent experiments.

TABLE I: Impact of the training view selection strategy (uniform versus random) on average visual quality. Evaluated under the fine-tuning regime with $N = 10$ views on the fern scene.

Model	Split	PSNR ↑	SSIM ↑	LPIPS ↓	ΔPSNR
PixelNeRF	Uniform	6.98	0.047	0.846	**+0.54**
	Random	6.44	0.042	0.840	
GNT	Uniform	10.34	0.179	0.738	+0.15
	Random	10.19	0.184	0.742	
Instant-NGP	Uniform	**13.57**	**0.231**	**0.627**	+0.06
	Random	13.51	0.229	0.630	
Nerfacto	Uniform	12.04	0.227	0.809	+0.11
	Random	11.93	0.226	0.865	

Specific pipeline configurations were strictly enforced to guarantee stability and fair comparison. Notably, camera pose refinement was disabled for Nerfacto to keep all models on equal footing, as this feature is not natively supported by the other evaluated architectures. To minimize memory footprint and accelerate fine-tuning, mixed-precision training was enabled across all models. For hash-based models (Instant-NGP and Nerfacto), the multiscale hash table capacity was constrained to 2^{16}, with backgrounds set to white. Structural hyperparameters for GNT were locked to 48 coarse and importance samples per ray, a transformer depth of 8, and a network width of 64, following its original configuration [6].

B. Models & Training Regimes

The evaluation encompasses distinct architectural paradigms: generalizable models (PixelNeRF [5] and GNT [6]) and highly efficient per-scene baselines (Instant-NGP [4] and Nerfacto [2]).

Models were evaluated across three foundational regimes: (i) ZS (Zero-Shot): immediate inference utilizing pre-trained weights without any adaptation; (ii) FT (Fine-Tuning): optimizing pre-trained generalizable weights directly on sparse target views ($N \in \{3, 6, 10\}$); and (iii) Per-Scene: optimizing baseline models from scratch under identical view constraints.

1) Fine-Tuning Strategies and Implementation: While Section **??** establishes the theoretical principles of fine-tuning regimes, their realization in generalizable NeRFs necessitates dedicated architectural hooks and parameter management logic. We implemented the five fine-tuning strategies within our custom Nerfstudio wrapper packages (`nerfstudio-gnt` and `nerfstudio-pixelnerf`) via a unified adaptation module (`adaptation.py`), decoupling strategy dispatch from the underlying volumetric rendering graph.

Parameter isolation is enforced dynamically at model initialization through `setup_adaptation()` by configuring the `requires_grad` property of sub-network parameter tensors and substituting targeted projection layers, as formalized in Algorithm 1. In Full Fine-Tuning (*Full-FT*), gradient propagation is enabled uniformly across all parameters of the 2D feature extractor and the volumetric rendering network. In Partial Fine-Tuning of the Decoder (*PFT-Decoder*), the spatial extractor (`feature_net` in GNT, `encoder` in PixelNeRF) has all parameters frozen (`p.requires_grad = False`) and is pinned to evaluation mode (`.eval()`), confining updates strictly to the rendering network (`net_coarse` and `net_fine` transformers in GNT, `mlp_coarse` and `mlp_fine` in PixelNeRF). In dissonance with the other investigated approaches, Partial Fine-Tuning of the Backbone (*PFT-Backbone*) inverts this configuration by granting gradients solely to the 2D extractor while freezing all rendering modules.

Selective PEFT (*BitFit*) freezes all multi-dimensional weight tensors throughout the network, deploying `apply_bitfit()` to recursively isolate and activate gradient computation exclusively on one-dimensional additive bias vectors (`name.endswith(".bias")`). This restricts optimization to under 0.15% of total network capacity without structural alterations. Additive PEFT (*PEFT-LoRA*) freezes the base model entirely, after which `apply_lora_to_module()` traverses the volumetric synthesis network to dynamically hot-swap linear projections (`q_proj`, `v_proj`, and dense MLP layers) with custom `LoRALinear` modules. Each adapter injects trainable down-projection $A \in \mathbb{R}^{r \times d_{\text{in}}}$ (initialized via Kaiming uniform) and up-projection $B \in \mathbb{R}^{d_{\text{out}} \times r}$ (initialized to zero) tensors, scaled by α/r.

Integrating these selective updates with Nerfstudio's multi-group optimizer engine required resolving empty-gradient exceptions. Nerfstudio queries `get_param_groups()` to register distinct learning rates for the 2D extractor and volumetric networks. When a sub-network is completely frozen, passing empty parameter lists causes standard PyTorch optimizers to fail. Our wrapper dynamically inspects each group and substitutes an inert scalar parameter (`dummy_param`) with `requires_grad = False` whenever no trainable tensors exist, preventing optimization crashes. Furthermore, to prevent PyTorch's top-level `model.train()` toggle from reinstating batch normalization drift in frozen extractors, `train()` is overridden to maintain `.eval()` on inactive submodules. Finally, to eliminate runtime latency during evaluation, `merge_lora_weights()` folds the accumulated low-rank update $\Delta W = \frac{\alpha}{r} BA$ directly into the base weights W_0 in-place prior to test-view synthesis, preserving native inference speed.

2) Stratified K-Fold Grid Search Protocol: Hyperparameter optimization and seed robustness were assessed using a

Algorithm 1 Dynamic Parameter Isolation and Fine-Tuning Setup

Require: Pre-trained model $\mathcal{M} = (\mathcal{F}_{\text{enc}}, \mathcal{R}_{\text{dec}})$ with parameters $\Theta = \Theta_{\text{enc}} \cup \Theta_{\text{dec}}$, strategy S, LoRA rank r, scaling α, target modules $\mathcal{T}$

Freeze all model parameters: $\nabla_\theta \leftarrow \text{False}, \forall \theta \in \Theta$
if $S =$ Zero-Shot **then**
  Set $\mathcal{F}_{\text{enc}}$.eval() {No gradient updates}
else if $S =$ Full-FT **then**
  $\nabla_\theta \leftarrow \text{True}, \forall \theta \in (\Theta_{\text{enc}} \cup \Theta_{\text{dec}})$
else if $S =$ PFT-Decoder **then**
  Set $\mathcal{F}_{\text{enc}}$.eval() and keep Θ_{enc} frozen
  $\nabla_\theta \leftarrow \text{True}, \forall \theta \in \Theta_{\text{dec}}$ {Tune rendering head only}
else if $S =$ PFT-Backbone **then**
  $\nabla_\theta \leftarrow \text{True}, \forall \theta \in \Theta_{\text{enc}}$ {Tune 2D spatial backbone only}
  Keep Θ_{dec} frozen
else if $S =$ BitFit **then**
  for all $\theta \in \Theta$ **do**
    if is_bias(θ) **then**
      $\nabla_\theta \leftarrow$ True {Activate 1D additive bias terms}
    end if
  end for
else if $S =$ PEFT-LoRA **then**
  Set $\mathcal{F}_{\text{enc}}$.eval()
  for all $W_0 \in \mathcal{R}_{\text{dec}}$ matching $\mathcal{T}$ **do**
    Replace W_0 with adapter: $W_0 x + \frac{\alpha}{r}(xA^T)B^T$
    Initialize $A \sim \mathcal{U}, B \leftarrow \mathbf{0}$
  end for
end if
Filter active parameter groups: $\mathcal{G} \leftarrow \text{FilterTrainable}(\Theta, \theta_{\text{dummy}})$

Ensure: Configured model $\mathcal{M}$ and optimizer parameter groups $\mathcal{G}$

Stratified K-Fold cross-validation protocol ($K = 4$) across all eight LLFF scenes. Each fold paired a high-frequency foliage scene with a structured architectural capture: Fold 0 (*fern*, *fortress*), Fold 1 (*flower*, *horns*), Fold 2 (*leaves*, *room*), and Fold 3 (*orchids*, *trex*). For every adaptation strategy, the search grid spanned three learning rate magnitudes $\eta \in \{10^{-4}, 5 \times 10^{-4}, 10^{-3}\}$ and two LoRA ranks $r \in \{4, 8\}$ (with $\alpha = 2r$), replicated across three independent random seeds $\{42, 123, 999\}$ under $N = 3$ input views.

To establish a calibrated iteration budget for this grid search, an empirical ablation over fine-tuning steps was conducted across the range $\{0, 100, 250, 500, 1000, 1500, 2000, 2500\}$, as depicted in Figure 2. As observed in Figure 2(a), PixelNeRF under *PFT-Backbone* rapidly ascends from 5.70 dB to peak visual quality at exactly 500 steps (11.03 dB PSNR, 0.150 SSIM). Beyond 500 steps, the model undergoes pronounced overfitting under the sparse three-view supervision, degrading to 8.57 dB at 2500 steps with a corresponding rise in perceptual loss (Figure 2(c)). For GNT under *PEFT-LoRA*, optimization advances monotonically, capturing the majority of its adaptation gain (14.29 dB, $+4.10$ dB over zero-shot) within the first 500 steps before plateauing towards 14.99 dB at 2000 steps. Consequently, 500 fine-tuning steps was selected as the rigorous standard for all subsequent grid search experiments, maximizing PixelNeRF's fidelity prior to geometric breakdown while capturing over 90% of GNT's asymptotic performance within an efficient computational window. Optimization runs encountering numerical breakdown due to dense occlusion geometry (yielding 0.0 dB PSNR on *leaves* and *orchids*) were systematically pruned to prevent distorting aggregate statistics.

TABLE II: Impact of N training views on PSNR (dB) on the room scene.

Model	Regime	N=3 ↑	N=6 ↑	N=10 ↑
PixelNeRF	ZS	7.35	7.15	7.29
	FT	7.24	7.26	7.13
GNT	ZS	**12.96**	**12.85**	**12.85**
	FT	11.78	**12.85**	11.58

TABLE III: PSNR (dB) under varying learning rates (η). Ablation conducted on the fern scene under the fine-tuning regime with $N = 10$ views.

Model	Regime	Learning Rate (η)	PSNR ↑
PixelNeRF	FT	10^{-4}	11.19
	FT	5×10^{-4}	12.23
	FT	10^{-3}	9.42
	ZS	N/A	6.64
GNT	FT	10^{-4}	**12.96**
	FT	5×10^{-4}	**12.96**
	FT	10^{-3}	12.85
	ZS	N/A	10.73

Standardizing the experimental framework required integrating all evaluated models into the Nerfstudio platform [2]. Although Instant-NGP and Nerfacto are natively supported, PixelNeRF and GNT required custom wrapper implementations, which we developed and released publicly. These modules map each model's native inference logic directly to Nerfstudio's ray generators and field architectures, incorporating tiny-cuda-nn [25] bindings for hardware acceleration.

Training used the Adam optimizer without weight decay, processing mini-batches of 512 rays during training and 128 rays during evaluation. Preliminary regime baselines ran for 2,500 iterations, the multi-seed fine-tuning grid search executed for 500 steps, and per-scene models were trained from scratch to convergence over 30,000 iterations. Although per-architecture tuning of η could yield higher peak metrics (Table III), systematic grid search was conducted to map optimal learning rates per strategy.

The complete wrapper codebase is available at https://github.com/Felipe-gsilva/Few-Shot-NeRFs/ and distributed as standalone packages via PyPI and PiWheels (https://www.piwheels.org/project/nerfstudio-gnt/ and https://www.piwheels.org/project/nerfstudio-pixelnerf/).

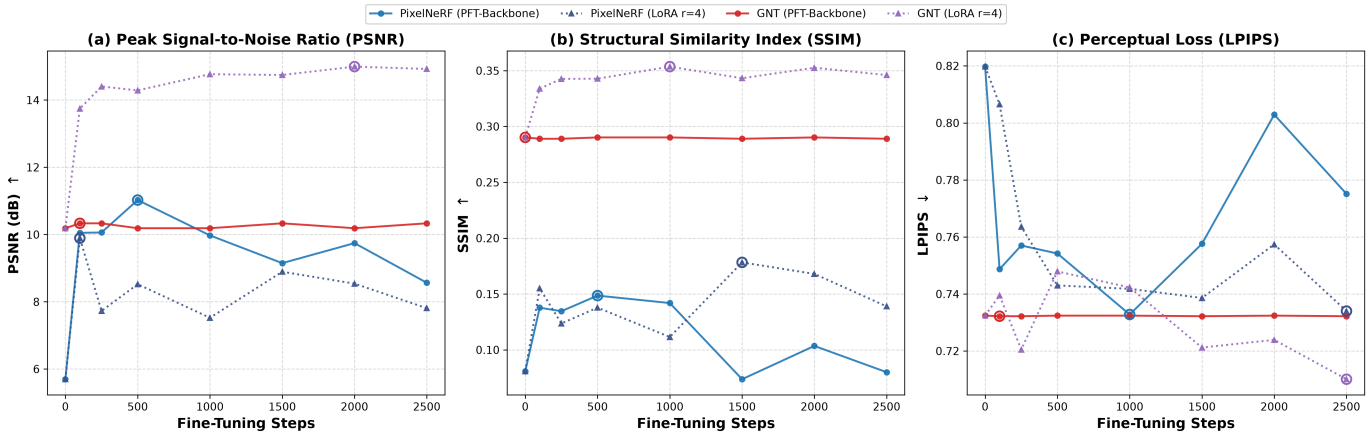


Fig. 2: Ablation on the number of fine-tuning steps under $N = 3$ input views. (a) PSNR, (b) SSIM, and (c) LPIPS across fine-tuning iterations for PixelNeRF and GNT under their leading adaptation configurations. Circled points denote individual optimums. PixelNeRF attains its global performance peak at 500 steps followed by representation collapse and overfitting, validating 500 steps as the calibrated iteration budget for the benchmarking grid search.

Benchmark scripts, configurations, and raw experimental logs are hosted at https://github.com/Felipe-gsilva/nerf-ann-paper/.

C. Evaluation & Metrics

The evaluation protocol assesses three complementary performance dimensions: visual fidelity, operational speed, and memory consumption. All reported metrics derive exclusively from the held-out test views (Section III-A). Visual reconstruction quality is quantified through Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index Measure (SSIM) [13], and Learned Perceptual Image Patch Similarity (LPIPS) [14]. In accordance with Niemeyer et al. [10], LPIPS employs VGG feature extractors to evaluate perceptual distance. Computational efficiency is assessed via rendering throughput in frames per second (FPS) over the test views [12]. Memory demands are characterized by static model parameter footprints alongside peak dynamic VRAM allocation recorded by PyTorch during inference. Execution within Nvidia Docker containers guaranteed environmental isolation and reproducible hardware telemetry on an NVIDIA GeForce GTX 1660 Super GPU (6GB VRAM, CUDA 12.6).

IV. Results and Discussion

This section presents the consolidated quantitative, computational and qualitative analyses across the evaluated LLFF scenes. Table IV summarizes the global averages for visual fidelity, temporal efficiency, and memory footprint across the architectural regimes. The results reveal critical trade-offs between generalization, optimization stability, and hardware constraints.

TABLE IV: Consolidated quantitative results across the LLFF dataset. Metrics represent global averages across valid test scenes. Training time reflects the best fine-tuning configuration for generalizable models and full convergence for Per-Scene models. VRAM Peak denotes dynamic memory allocation during inference.

Model	Regime	PSNR ↑	SSIM ↑	LPIPS ↓	Time ↓	FPS ↑	VRAM ↓
PixelNeRF	ZS	6.35	0.085	0.800	∼22 s	0.004	1.93
	FT	10.92	0.162	0.772	4 min 36 s	0.006	1.97
GNT	ZS	10.51	0.314	0.742	**∼9 s**	0.014	1.94
	FT	15.07	0.368	0.679	4 min 59 s	0.015	1.93
Instant-NGP	N=10	14.06	0.442	0.498	31 min 51 s	0.199	**1.09**
Nerfacto	N=10	**27.11**	**0.875**	**0.171**	8 min 57 s	**0.388**	1.43

A. Visual Fidelity

The per-scene architectures demonstrated vastly superior visual reconstruction capabilities under dense optimization. Nerfacto established the performance upper bound, achieving a global average of 27.11 dB PSNR and 0.875 SSIM (Table IV). In contrast, the generalizable models exhibited marked differences in their capacity to exploit structural priors versus fine-tuning under sparse view constraints ($N \leq 10$).

In purely zero-shot (ZS) inference, PixelNeRF produced degenerate reconstructions across all LLFF scenes (6.35 dB global average, Table V), since its structural priors were insufficient for accurate depth triangulation under sparsity. In contrast, GNT achieved better performative zero-shot reconstructions (10.51 dB, Table V) leveraging its cross-attention transformer to aggregate global geometric priors even though the images were not scructurally representative.

When fine-tuned on the sparse novel views, both generalizable models demonstrated substantial fidelity gains under appropriate parameter-efficient strategies, overcoming the premature representational collapse observed under naive full fine-tuning. As shown in Table IV, GNT reached 15.07

dB PSNR (and up to 17.46 dB on structured scenes such as *fortress*), surpassing Instant-NGP's sparse-view baseline (14.06 dB). Similarly, PixelNeRF recovered from near-total failure to 10.92 dB PSNR under backbone-specific fine-tuning (Table V).

Furthermore, the stability of these visual results across different scenes is illustrated in Figure 3. The boxplot highlights the significant performance variance inherent to each paradigm. While per-scene models exhibit sensitivity to geometric scene complexity (with Instant-NGP ranging from 19.33 dB down to 13.51 dB depending on texture), generalizable models exhibit tight inter-scene variance in zero-shot mode that expands upon fine-tuning as models adapt to high-frequency scene details.

B. Comparative Analysis of Fine-Tuning Strategies

Table V compiles the consolidated comparative results across the five fine-tuning strategies evaluated on PixelNeRF and GNT. The results are aggregated across 3 independent random seeds $(42, 123, 999)$ and all valid LLFF scenes, strictly pruning failed zero-PSNR convergence runs.

TABLE V: Comparative analysis of fine-tuning strategies: PixelNeRF (5 valid scenes) vs. GNT (6 valid scenes). Metrics represent multi-seed aggregation across 3 independent runs (seeds 42, 123, 999). Failed optimization runs (zero PSNR) were filtered out.

Model	Strategy	η	r	PSNR (dB) ↑	SSIM ↑	LPIPS ↓
PixelNeRF	Zero-Shot (ZS)	N/A	N/A	6.35 ± 0.05	0.085	0.800
	Full-FT	10^{-4}	N/A	9.03 ± 0.11	0.136	0.752
	PFT-Decoder	10^{-4}	N/A	10.58 ± 0.08	0.205	0.773
	PFT-Backbone	10^{-4}	N/A	**10.92** ± 0.26	0.162	0.772
	BitFit	10^{-4}	N/A	10.79 ± 0.19	0.119	0.775
	PEFT-LoRA	10^{-3}	4	8.59 ± 0.10	0.175	0.725
GNT	Zero-Shot (ZS)	N/A	N/A	10.51 ± 0.11	0.314	0.742
	Full-FT	10^{-3}	N/A	15.06 ± 0.03	**0.369**	0.681
	PFT-Decoder	10^{-3}	N/A	**15.07** ± 0.03	0.368	**0.679**
	PFT-Backbone	10^{-4}	N/A	10.51 ± 0.11	0.314	0.742
	BitFit	10^{-3}	N/A	13.12 ± 0.05	0.358	0.742
	PEFT-LoRA	10^{-3}	4	14.57 ± 0.07	**0.369**	0.706

The empirical evaluation reveals a clear architectural divergence in how generalizable backbones balance adaptation capacity and geometric stability during sparse-view fine-tuning.

For PixelNeRF, performance peaks when fine-tuning is restricted to the feature extraction backbone (*PFT-Backbone*, reaching 10.92 ± 0.26 dB) or to the additive bias parameters (*BitFit*, 10.79 ± 0.19 dB), substantially surpassing unconstrained *Full-FT* (9.03 ± 0.11 dB). Because PixelNeRF projects 2D features along camera rays, its primary bottleneck resides in extracting view-consistent representations from wide-baseline sparse cameras. Updating the ResNet backbone adjusts local feature maps to out-of-distribution textures and sensor profiles, whereas updating all weights simultaneously destabilizes the downstream MLP under sparse supervision. Moreover, PixelNeRF exhibits acute sensitivity to optimization rates, requiring conservative updates ($\eta = 10^{-4}$) to prevent catastrophic geometric divergence.

In contrast, GNT achieves its highest reconstruction fidelity under *PFT-Decoder* (15.07 ± 0.03 dB PSNR, 0.679 LPIPS), statistically matching *Full-FT* (15.06 ± 0.03 dB) while eliminating visual transformer optimizer memory overhead and reducing per-scene fine-tuning latency from 357.1 s down to 298.8 s (a 16.3% speedup). Because GNT already possesses robust global visual representations from pre-training, freezing its Vision Transformer backbone prevents representation drift and directs gradient descent exclusively into the epipolar cross-attention layers to resolve scene-specific radiance integration. In stark contrast, *PFT-Backbone* yields 10.51 ± 0.11 dB—virtually indistinguishable from zero-shot inference—confirming that GNT's domain adaptation is governed almost entirely by its epipolar transformer rather than low-level patch projections.

Consistent with observations reported across foundation models in other modalities [24], selective and low-rank PEFT demonstrate remarkable stability under sparse supervision. In GNT, *BitFit* reaches 13.12 ± 0.05 dB (+2.61 dB over zero-shot) by updating solely additive bias vectors (fewer than 0.5% of parameters), while *PEFT-LoRA* ($r = 4$) delivers 14.57 ± 0.07 dB (+4.06 dB over zero-shot) and optimal perceptual quality (SSIM 0.369). Restricting parameter trajectories to low-rank subspaces or bias manifolds regularizes optimization against ray-level overfitting, mitigating the floaters and collapse that plague unconstrained sparse-view fine-tuning.

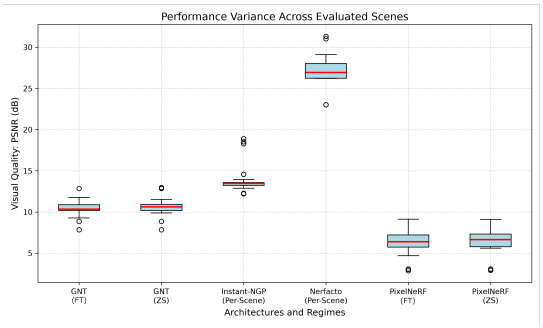


Fig. 3: Visual performance variance (PSNR) across all evaluated LLFF scenes.

C. Computational Trade-offs

From a systemic viability perspective, a clear dichotomy emerges between inference latency and training overhead. While generalizable models bypass extensive optimization (requiring only seconds for ZS inference or $\sim$7 minutes for FT), their volumetric feature aggregation mechanisms result in catastrophic inference bottlenecks, yielding rendering speeds below 0.02 FPS.

Conversely, the explicit multiscale hash representations of the per-scene models proved vastly more efficient during runtime. Despite requiring lengthier convergence periods (from $\sim$9 minutes for Nerfacto up to $\sim$32 minutes for Instant-NGP), they achieve inference rates ranging from $13\times$ (Instant-NGP vs. GNT) to $97\times$ (Nerfacto vs. PixelNeRF) faster than the generalizable baselines. Furthermore, the hash-based models

exhibit a highly optimized dynamic memory footprint. While Instant-NGP operates at just 1.10 GB of VRAM during inference on the fern scene (Table VI), the global average footprint across all scenes scales slightly to 1.43 GB for Nerfacto (Table IV) due to structural complexity variance between captures. Nonetheless, these metrics solidify per-scene methods as the most viable path for hardware-constrained deployments despite the initial training cost.

D. Regime Trade-off Synthesis

The architectural trade-offs evaluated establish a rigid Pareto frontier between data availability and computational overhead, as visually summarized by the multidimensional radar chart in Figure 4 . When multi-view data is abundant but inference latency and VRAM are strictly bounded, the per-scene paradigm (specifically Nerfacto) dominates the design space, delivering state-of-art fidelity with sub-second rendering. Conversely, under view scarcity ($N \leq 10$), the structural priors of generalizable models become indispensable to prevent total geometric collapse, yet they exact a severe penalty: inference speeds plummet to impractical levels (≤ 0.02 FPS) and visual quality plateaus prematurely, as the frozen feature extractor cannot adapt its priors to scene-specific geometric complexities (a manifestation of the representational bottleneck described in Section II). Thus, the selection between regimes is not absolute, but entirely dictated by whether the deployment bottleneck lies in camera acquisition physics or rendering hardware capacity.

TABLE VI: Computational Performance and Memory on the fern scene.

Model	Regime	Time (s)	Inference (FPS)	VRAM Peak (GB)
PixelNeRF	ZS	39.02	0.003	1.60
	FT	496.82	0.006	1.92
GNT	ZS	**8.77**	0.007	1.97
	FT	392.15	0.008	1.97
Instant-NGP	N=10	1707.91	0.183	**1.09**
Nerfacto	N=10	423.45	**0.371**	1.10

E. Qualitative Analysis

The qualitative assessment visually corroborates the quantitative metrics, specifically illustrating the boundaries of each architectural paradigm. For visual clarity, novel views matching the exact poses from the test set were synthesized using perspective projection.

Figure 5 highlights the disparities in reconstruction quality on the highly detailed fern scene. As visually evident, neither PixelNeRF nor GNT achieved proper convergence. PixelNeRF produced renderings with intense isotropic blurring, a typical artifact indicating the network's failure to accurately localize 2D projected features across extremely sparse views. GNT, on the other hand, demonstrated a complete generalization failure regarding the in-the-wild characteristics of the scene. This resulted in a representational visual collapse, heavily distorting the volumetric geometry into noise, which persisted even after 10-view fine-tuning.

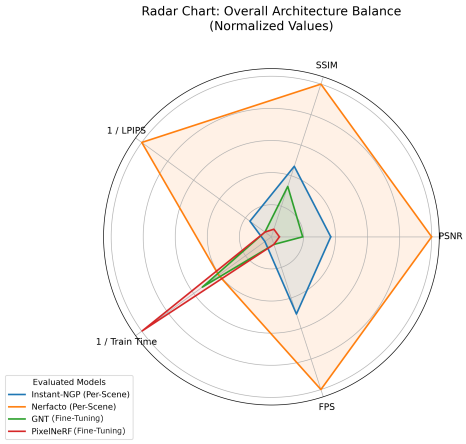


Fig. 4: Multidimensional trade-offs under sparse-view constraints ($N = 10$). Metrics are max-normalized (outer perimeter = optimal), utilizing $1/\text{LPIPS}$ and $1/\text{Train Time}$ for uniform directionality. Per-scene models dominate fidelity and inference speed, whereas generalizable methods strictly excel in optimization efficiency.

Conversely, the per-scene architectures successfully captured the intricate foliage geometry. Nerfacto achieved the highest visual fidelity across all scenes (PSNR 27.11 dB, SSIM 0.875, LPIPS 0.171), recovering fine-grained details consistent with the ground truth resolution (Figure 5, bottom right).

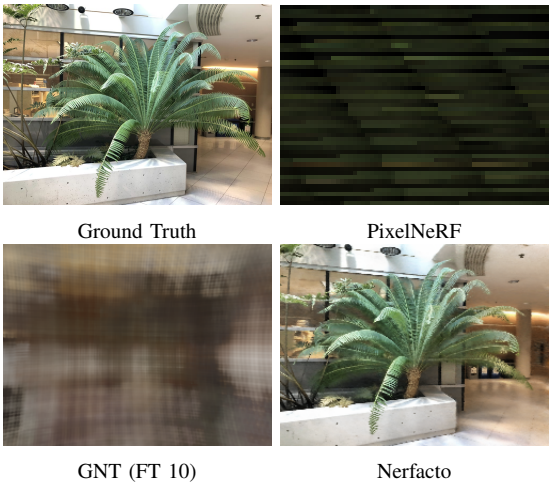


Fig. 5: Qualitative comparison on the fern scene. Generalizable architectures (PixelNeRF and GNT) failed to converge under sparse constraints, resulting in severe blurring and visual collapse. In contrast, per-scene optimization models (specifically Nerfacto) successfully reconstructed the complex geometry.

V. Conclusion

This work developed a methodology for systematically characterizing the trade-offs between computational performance and reconstruction fidelity of Generalizable and Per-Scene

NeRF architectures under sparse-view conditions. The experimental results demonstrate that Per-Scene models achieve substantially lower inference latency and VRAM consumption than generalizable models, making them well suited for deployment in resource-constrained environments. However, these benefits rely on dense multi-view supervision, restricting their usage in sparse-view acquisition scenarios, where generalizable architectures remain a practical alternative despite reduced reconstruction fidelity.

Although the proposed strategy provides a standardized and reproducible framework, some limitations should be acknowledged. The restricted hardware configuration of the experimental environment prevented the inclusion of architectures that rely on computationally intensive 3D cost volumes or strong geometric regularization, such as RegNeRF. Furthermore, the experimental validation was conducted on the LLFF dataset, and the adopted sparse-view sampling strategy exploits the sequential ordering of its image indices, which may limit its direct applicability to unordered image collections.

In this context, future work will focus on extending the proposed methodology to unbounded 360-degree datasets and more diverse acquisition scenarios, enabling a broader characterization of these trade-offs. This includes incorporating sparse-view 3D Gaussian Splatting variants [26], [27] into the benchmarking protocol, to assess whether their explicit representation offers a more favorable fidelity–efficiency trade-off than the implicit architectures evaluated here. Also, diffusion priors will be investigated as regularization mechanisms during FT to improve reconstruction quality.

ACKNOWLEDGMENT

The authors gratefully acknowledge the financial support of the National Council for Scientific and Technological Development - CNPq (Grants #305386/2024-7, #302833/2025-0, #307318/2022-2 and #404571/2025-5), the State of Minas Gerais Research Foundation - FAPEMIG (Grant #APQ-00727-24 and #APD-01428-25) and the State of São Paulo Research Foundation - FAPESP (Grant #2022/03020-1).

REFERENCES

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023. [Online]. Available: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/

[2] M. Tancik, E. Weber, E. Ng, R. Li, B. Yi, T. Wang, A. Kristoffersen, J. Austin, K. Salahi, A. Ahuja, D. Mcallister, J. Kerr, and A. Kanazawa, "Nerfstudio: A modular framework for neural radiance field development," in *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Proceedings*, ser. SIGGRAPH '23. ACM, Jul. 2023, p. 1–12. [Online]. Available: http://dx.doi.org/10.1145/3588432.3591516

[3] Adobe, *Substance 3D Viewer (beta)*, 2025, computer Software. [Online]. Available: https://helpx.adobe.com/substance-3d-viewer/get-started/quick-start.html

[4] T. Müller, A. Evans, C. Schied, and A. Keller, "Instant neural graphics primitives with a multiresolution hash encoding," *ACM Transactions on Graphics*, vol. 41, no. 4, p. 1–15, Jul. 2022. [Online]. Available: http://dx.doi.org/10.1145/3528223.3530127

[5] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, "pixelnerf: Neural radiance fields from one or few images," 2021. [Online]. Available: https://arxiv.org/abs/2012.02190

[6] Z. Wang, M. Varma, P. Wang, X. Chen, T. Chen, and S. Venugopalan, "Is attention all that nerf needs?" 2023. [Online]. Available: https://arxiv.org/abs/2207.13298

[7] K. Schwarz, Y. Liao, M. Niemeyer, and A. Geiger, "Graf: Generative radiance fields for 3d-aware image synthesis," 2021. [Online]. Available: https://arxiv.org/abs/2007.02442

[8] H. Wu, B. Xiao, N. Codella, M. Liu, X. Dai, L. Yuan, and L. Zhang, "Cvt: Introducing convolutions to vision transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2021, pp. 22–31.

[9] P. Hedman, P. P. Srinivasan, B. Mildenhall, J. T. Barron, and P. Debevec, "Baking neural radiance fields for real-time view synthesis," 2021. [Online]. Available: https://arxiv.org/abs/2103.14645

[10] M. Niemeyer, J. T. Barron, B. Mildenhall, M. S. Sajjadi, A. Geiger, and N. Radwan, "Regnerf: Regularizing neural radiance fields for view synthesis from sparse inputs," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 5480–5490.

[11] H. Heo, T. Kim, J. Lee, J. Lee, S. Kim, H. Kim, and J.-H. Kim, "Robust camera pose refinement for multi-resolution hash encoding," 02 2023.

[12] J. Kulhanek and T. Sattler, "Nerfbaselines: Consistent and reproducible evaluation of novel view synthesis methods," 2025. [Online]. Available: https://arxiv.org/abs/2406.17345

[13] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.

[14] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 586–595.

[15] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," 2020. [Online]. Available: https://arxiv.org/abs/2003.08934

[16] A. Laurino, A. Franceschini, L. Pesce, L. Cinci, A. Montalbano, G. Mazzamuto, G. Sancataldo, G. Nesi, I. Costantini, L. Silvestri, and F. S. Pavone, "A guide to perform 3d histology of biological tissues with fluorescence microscopy," *International Journal of Molecular Sciences*, vol. 24, no. 7, 2023. [Online]. Available: https://www.mdpi.com/1422-0067/24/7/6747

[17] A. Jain, M. Tancik, and P. Abbeel, "Putting NeRF on a diet: Semantically consistent few-shot view synthesis," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 5885–5894.

[18] A. Chen, Z. Xu, F. Zhao, X. Zhang, F. Xiang, J. Yu, and H. Su, "Mvsnerf: Fast generalizable radiance field reconstruction from multi-view stereo," 2021. [Online]. Available: https://arxiv.org/abs/2103.15595

[19] Q. Wang, Z. Wang, K. Genova, P. P. Srinivasan, H. Zhou, J. T. Barron, R. Martin-Brualla, N. Snavely, and T. Funkhouser, "IBRNet: Learning multi-view image-based rendering," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 4690–4699.

[20] B. Mildenhall, P. P. Srinivasan, R. Ortiz-Cayon, N. K. Kalantari, R. Ramamoorthi, R. Ng, and A. Kar, "Local light field fusion: Practical view synthesis with prescriptive sampling guidelines," 2019. [Online]. Available: https://arxiv.org/abs/1905.00889

[21] B. Roessle, J. T. Barron, B. Mildenhall, P. P. Srinivasan, and M. Nießner, "Dense depth priors for neural radiance fields from sparse input views," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 892–12 901.

[22] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations (ICLR)*, 2022.

[23] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022, pp. 1–9.

[24] D. Gupta, A. Bhatti, S. Parmar, C. Dan, Y. Liu, B. Shen, and S. Lee, "Beyond LoRA: Exploring efficient fine-tuning techniques for time series foundational models," *arXiv preprint arXiv:2409.11302*, 2024.

[25] T. Müller, "tiny-cuda-nn," https://github.com/NVlabs/tiny-cuda-nn, 2021, version 1.7, BSD-3-Clause License.

[26] K. Seo, S. Hyun, M. Lee, and J.-P. Heo, "Improving sparse-view 3dgs generalization via flat minima optimization," 2026. [Online]. Available: https://arxiv.org/abs/2607.00885

[27] M. Song, X. Lin, D. Zhang, H. Li, X. Li, B. Du, and L. Qi, "D^2gs: Depth-and-density guided gaussian splatting for stable and accurate sparse-view reconstruction," 2025. [Online]. Available: https://arxiv.org/abs/2510.08566